Submitted by,

Course Title: Object-Oriented Programming (Java)

Course Code: CSE 2104

Lab Report: 01

Student Name: Mahinur Rahman Mahin

Student ID: 242014165

Section: 01

Department: CSE

University of Liberal Arts Bangladesh (ULAB)

Submitted to,

Fatema Khan, Lecturer

Department of Computer Science & Engineering (CSE)

University of liberal arts Bangladesh

1st Problem -

1. Problem Title: Bank Account Management System using the class and object.

2. Objective: Implementing deposit and withdrawal operations using methods.

3. Problem Description: A creation of a class known as BankAccount with instance variables that include account number and balance.

4. Algorithm:

Step 1. Start the program.

Step 2. Scanner class for user input.

Step 3. Class named BankAccount.

Step 4. Instance variables accountNumber and balance.

Step 5. Create a constructor to initialize account details.

Step 6. Create a deposit() method to add money to the account balance.

Step 7. Create a withdraw() method to subtract money from the account balance if sufficient funds are available.

Step 8. Create a display() method to show account information.

Step 9. Account number and initial balance from the user.

Step 10. BankAccount the constructor.

Step 11. Display the initial account information.

Step 12. Take withdrawal amount.

Step 13. Display the updated account information.

Step 14. End the program.

5. Source Code:

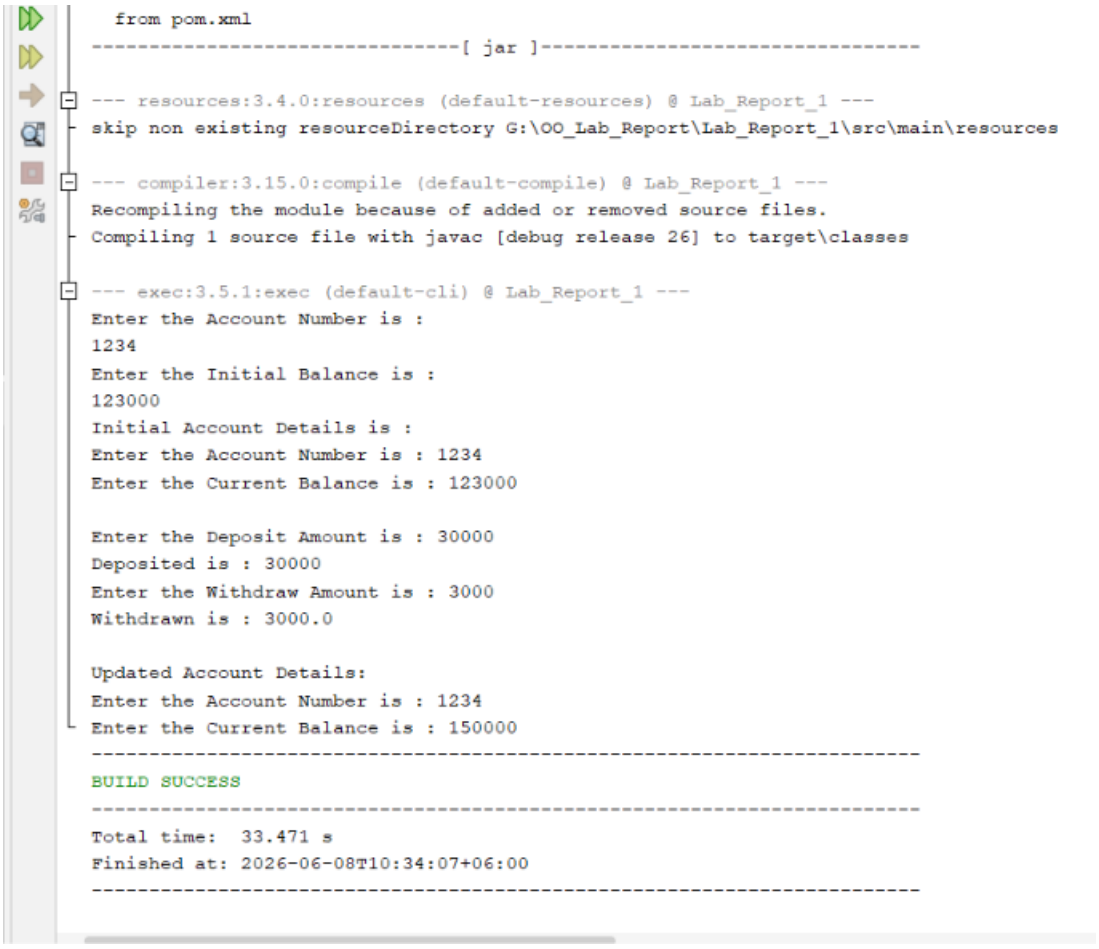
```
package com.mycompany.lab_report_1 ;
import java.util.Scanner  ;
public class BankAccount
{
    String Account_Number ;
    long Balance ;
    public BankAccount (String Account_Number , long Balance)
```

```
{
    this.Account_Number = Account_Number ;
    this.Balance = Balance ;
}
 // Deposit
 public void deposit ( long Amount)
{
    if (Amount > 0 )
    {
        Balance += Amount ;
        System.out.println ("Deposited is : " +Amount)   ;
    }
    else
     {
        System.out.println ("Deposit amount must be positive.") ;
     }
}
 // Withdraw
public void withdraw (double Amount)
{
    if ( Amount > 0 && Amount <= Balance )
    {
        Balance -= Amount ;
        System.out.println ("Withdrawn is : " +Amount) ;
    }
    else if (Amount > Balance)
    {
         System.out.println ("Insufficient Balance.") ;
    }
    else
```

```
        {
            System.out.println ("Withdraw amount must be positive." ) ;
        }
}
// Display
    public void display()
    {
        System.out.println ("Enter the Account Number is : " + Account_Number )   ;
        System.out.println ("Enter the Current Balance is : " + Balance ) ;
    }
    public static void main (String[] args)
    {
        Scanner s1 = new Scanner (System.in ) ;
        System.out.println("Enter the Account Number is : " ) ;
        String account_number = s1.nextLine() ;
        System.out.println ("Enter the Initial Balance is : ") ;
        long initBalance = s1.nextLong() ;
        BankAccount Account = new BankAccount (account_number , initBalance ) ;
        System.out.println ("Initial Account Details is : ") ;
        Account.display() ;
        System.out.print("\nEnter the Deposit Amount is : ") ;
        long depositAmount = s1.nextLong() ;
        Account.deposit(depositAmount) ;
```

```
        System.out.print("Enter the Withdraw Amount is : ") ;
        long withdrawAmount = s1.nextLong() ;
        Account.withdraw(withdrawAmount) ;
        System.out.println("\nUpdated Account Details:") ;
        Account.display() ;
        s1.close() ;
    }
}
```

6. Output:



7. Explanation:

Classes were used because: BankAccount class was used for representing the bank account in real world. All related data members along with operations are grouped together forming a logical unit.

Methods were used for:

i. deposit(): To deposit some amount in the account.

ii. withdraw(): To withdraw some amount from the account.

iii. display(): To display the details of the account.

OOP concept applied:

i. Class and Object: BankAccount is a class.

ii. Constructor: initialize the data members of the object.

iii. Encapsulation: Data members combined together to form one logical entity known as class.

8. Conclusion: A creation of a class as BankAccount with instance variables. This program enables the user to input their bank account details, deposit into the account, withdraw the account, and display results.

2nd Problem -

Problem Title: Length and Width Properties and Calculate Area and Perimeter.

Objective:

i. To understand the concept of classes and objects in Java.

ii. To learn how constructors initialize object properties.

iii. Create a method to calculate the area of Rectangle avolution problem description Create a Rectangle class with two attributes length and width.

Problem Description: A Movie object gets made holding data like title, genre, main actor, director, release year, score, and critique. One film has its info typed by the person. The critique forms itself from the movie score value. Should the score fall below

five, the critique reads Not Good; if higher, it says Good. At end, details for both films get shown out.

Algorithm

Step 1. Start the program.

Step 2. Import the Scanner class.

Step 3. Rectangle class with length and width attributes.

Step 4. Constructor to initialize length and width.

Step 5. Method nameed calculateArea().

Step 6. CalculatePerimeter() to calculate the perimeter.

Step 7. Create a display().

Step 8. To take length and width as input from the user.

Step 9. Create a Rectangle object using the input values.

Step 10. Call the display().

Step 11. Close the Scanner object.

Step 12. End the program.

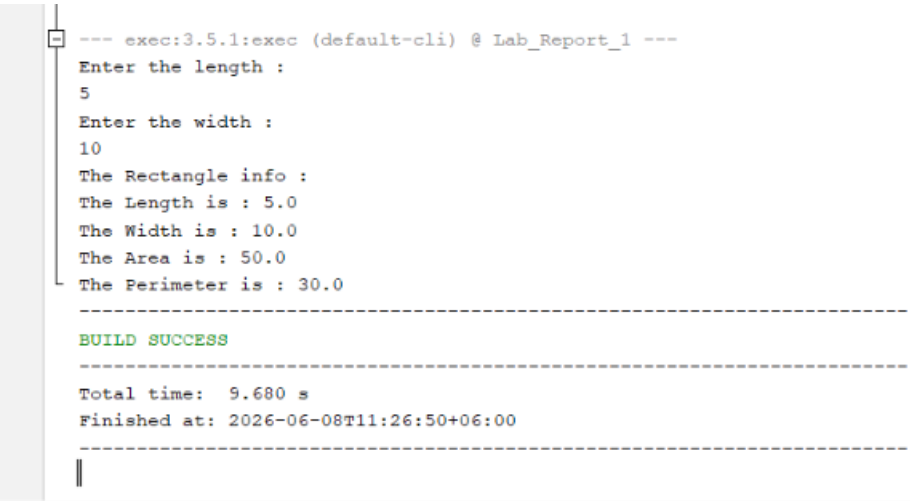
Source Code:

```
package com.mycompany.lab_report_1;
import java.util.Scanner ;
public class Rectangle
{
   double length ;
   double width  ;
   Rectangle ( double length , double width )
   {
      this.length = length ;
      this.width = width ;
   }
   double Area()
```

```
    {
        return length * width ;
    }
double Perimeter()
 {
        return 2 * ( length + width ) ;
    }
     void display()
    {
        System.out.println("The Length is : " + length ) ;
        System.out.println("The Width is : " + width ) ;
        System.out.println("The Area is : " + Area()  ) ;
        System.out.println("The Perimeter is : " + Perimeter()  ) ;
    }
    public static void main (String[] args )
    {
        Scanner s1 = new Scanner (System.in  ) ;
        System.out.println ("Enter the length : " ) ;
        double length = s1.nextDouble() ;
        System.out.println ("Enter the width : ") ;
        double width = s1.nextDouble() ;
        Rectangle rectangle = new Rectangle (length , width ) ;
        System.out.println ("The Rectangle info : ") ;
        rectangle.display() ;
        s1.close () ;
```

```
    }
}
```

Output:



Explanation:

Why classes were applied?

Ans: In this case, the rectangle was implemented by a class that represents it called Rectangle. This is because we Carve out a data and behavior working as one.

Why Write These Methods like : Area() → area of the rectangle The Perimeter() method calculates the perimeter of a rectangle.

OOP Principle Applied:

i. Class & Objects Rectangle is a class

ii. Encapsulation : Properties and methods related to the rectangle together you have made a single rectangle class.

iii. Constructors: In the constructor, length and width values of objects are initialized.

Conclusions: This project help to learn.

3rd Problem -

Problem Title: Movies are Title, Genre, Lead Actor, Director, Release Year, Rating, and Review.

Objective: To take input from using the Scanner class.

Problem Description: A Movie class is created with properties such as title, genre, lead actor, director, release year, rating, and review included. The user inputs data for single movies. This review is automatically generated. The rating may change. If rating is less than 5 then review is “Not Good”. If not then review is “Good”.

Algorithm

Step 1. Start the program.

Step 2. Import the Scanner class.

Step 3. Create a Movie class with required properties.

Step 4. Create a constructor to initialize movie information.

Step 5. Check the movie rating:

i. If rating < 5, set review = "Not Good".

ii. Otherwise, set review = "Good".

Step 6. Create a display() method to show movie details.

Step 7. Take information for the first movie from the user.

Step 8. Create the first Movie object.

Step 9. Take information for the second movie from the user.

Step 10. Create the second Movie object.

Step 11. Display information of both movie objects.

Step 12. Close the Scanner object.

Step 13. End the program.

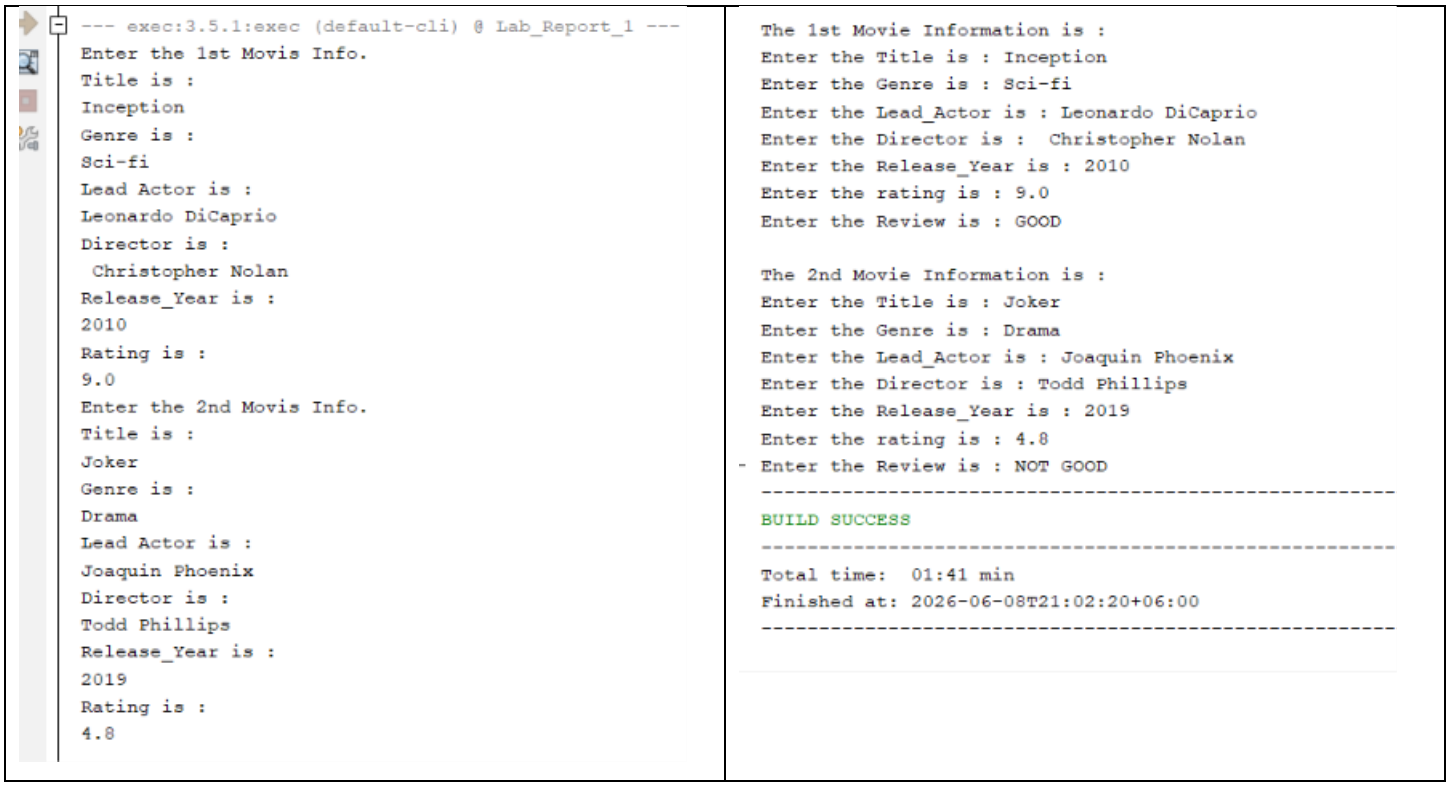
Source Code

```
package com.mycompany.lab_report_1;
import java.util.Scanner ;
```

```
public class Movie
{
    String title , genre , lead_actor , director , review ;
    int release_year ;
    double rating ;
    Movie ( String title , String genre , String Lead_Actor , String director , int
release_year , double rating )
{
        this.title = title ;
        this.genre = genre ;
        this.lead_actor = Lead_Actor ;
        this.rating = rating ;
        this.release_year = release_year ;
        this.director = director ;
        if ( rating <= 5 )
        {
            review = "NOT GOOD" ;
        }
       else
       {
          review = "GOOD" ;
       }
}
void display()
   {
        System.out.println("Enter the Title is : " + title ) ;
        System.out.println("Enter the Genre is : " + genre ) ;
        System.out.println("Enter the Lead_Actor is : " + lead_actor ) ;
        System.out.println("Enter the Director is : " + director ) ;
        System.out.println("Enter the Release_Year is : " + release_year ) ;
```

```
        System.out.println("Enter the rating is : " + rating ) ;
        System.out.println("Enter the Review is : " + review  ) ;
    }
    public static void main (String[] args )
    {
        Scanner s1 = new Scanner (System.in ) ;
        System.out.println("Enter the Movis Info."  ) ;
        System.out.println ("Title is : ") ;
        String title = s1.nextLine() ;
        System.out.println ("Genre is : ") ;
        String genre = s1.nextLine() ;
        System.out.println ("Lead Actor is : ") ;
        String lead_actor = s1.nextLine() ;
        System.out.println ("Director is : ") ;
        String Director = s1.nextLine() ;
        System.out.println ("Release_Year is : ") ;
        int release_year = s1.nextInt() ;
        System.out.println ("Rating is : ") ;
        double rating = s1.nextDouble() ;
        Movie movie = new Movie ( title , genre , lead_actor , Director , release_year ,
rating)
        System.out.println ("\nThe Movie Information is : ") ;
        movie.display() ;
        s1.close()  ;
    }
}
```

Output:



```
--- exec:3.5.1:exec (default-cli) @ Lab_Report_1 ---
Enter the 1st Movis Info.
Title is :
Inception
Genre is :
Sci-fi
Lead Actor is :
Leonardo DiCaprio
Director is :
 Christopher Nolan
Release_Year is :
2010
Rating is :
9.0
Enter the 2nd Movis Info.
Title is :
Joker
Genre is :
Drama
Lead Actor is :
Joaquin Phoenix
Director is :
Todd Phillips
Release_Year is :
2019
Rating is :
4.8
```

```
The 1st Movie Information is :
Enter the Title is : Inception
Enter the Genre is : Sci-fi
Enter the Lead_Actor is : Leonardo DiCaprio
Enter the Director is :  Christopher Nolan
Enter the Release_Year is : 2010
Enter the rating is : 9.0
Enter the Review is : GOOD

The 2nd Movie Information is :
Enter the Title is : Joker
Enter the Genre is : Drama
Enter the Lead_Actor is : Joaquin Phoenix
Enter the Director is : Todd Phillips
Enter the Release_Year is : 2019
Enter the rating is : 4.8
Enter the Review is : NOT GOOD
------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------
Total time:  01:41 min
Finished at: 2026-06-08T21:02:20+06:00
------------------------------------------------------------
```

Explanation:

Reason for Using Classes: The Movie class was used to represent movie data as a single object. It groups together related data and operations . This makes the program organized and reusable .

Why Methods Were Made: When an object is created, the constructor sets movie properties. The display() method shows all the info of the movie in a structured way.

Which OOP Concept Was Applied:

i. Class and Object — Movie : class movie1: objects

ii. Encapsulation: Movie data and behavior is encapsulated in a single class.

iii. Constructor: Initializing an object's movie its created.

Conclusions: From different classes, objects, constructors and methods, we learnt about conditional statements in Java. You learnt how to create multiple object, take user input and generate reviews as per rating using java. Utilizing all the brief learned this subject and enhance my understanding.